analyse this along with screenshots and animations and UI and screens and layouts and features: https://play.google.com/store/apps/details?id=com.ecffri.arrows&hl=en_IN

and i want to recreate this game,
can you give me a detailed prompt to generate this game?

i'll be using cursor / codex to build this

Show more

Worked for 9s

Yes. I checked the current Google Play listing, its screenshots, several gameplay references, and recent reviews. The current version describes the game as a minimalist logic puzzle where the player extracts arrows in the correct order without collisions; it also has hints, hearts, large handcrafted progression, evolving mechanics, offline play, and calming audio. Google Play +1

The screenshots reveal an especially important implementation detail: **the puzzle isn't simply a collection of straight arrows**. The arrows can form long, thin orthogonal/polyline shapes, with the arrowhead indicating the direction in which the entire piece escapes. The board progressively becomes denser and more difficult. Google Play +2

There are also later-state boards with colored/active arrows, hearts, and auxiliary visual guides. Google Play +1

And the current game has expanded beyond the basic puzzle loop: reviews mention **daily puzzles, historical daily puzzles, tournaments, streaks/trophies and difficulty such as Easy/Hard/Super Hard/Nightmare**. Google Play +1

So I would **not ask Cursor/Codex to "make an Arrows clone"**. I'd give it a detailed product/engineering specification for an **original arrow-escape puzzle game inspired by the mechanics**, with its own branding, assets and UI.

Below is the prompt I'd actually use.

MASTER CURSOR / CODEX PROMPT

Copy everything below into your coding agent as the **initial project specification**.

```
You are the lead game engineer, gameplay programmer, UI engineer, technical ar
puzzle algorithm designer and QA engineer for this project.

Build a complete production-quality mobile puzzle game inspired by the core me
modern arrow escape / arrow maze puzzle games.

IMPORTANT:
- Do NOT copy any copyrighted artwork, branding, logo, exact UI assets, sounds
  screenshots or proprietary level data from another game.
- Recreate the GAMEPLAY CONCEPT using original implementation, original visual
  original branding and original level data.
- The game should feel polished enough to ship.
- Prioritize deterministic gameplay and clean architecture over hacks.
- Do not create a fake prototype where buttons do nothing.
- Every visible feature must have a working implementation.
- The project must compile and run after every major milestone.

TARGET PLATFORM:
iOS.

TECHNOLOGY:
- Swift
- SpriteKit for the puzzle/game rendering layer
- SwiftUI for menus/settings/progression screens where appropriate
- Use UIKit only where it provides a clear advantage
- No third-party dependencies unless explicitly justified
- Local persistence using Codable + UserDefaults initially
- Design the architecture so persistence can later move to Core Data/SQLite/cl
- Support iPhone and iPad
- Portrait orientation
- Respect safe areas
- Support Dynamic Type where appropriate for non-game UI
- Support reduced motion and accessibility settings

PROJECT NAME:

"ArrowFlow"

This name is temporary and can be changed later.

CORE GAME CONCEPT
==================

The player sees a board containing multiple thin arrow-shaped pieces.

Each arrow consists of:
- a connected orthogonal path/body
- an arrowhead
- a movement direction
- occupied grid cells

The player must remove every arrow from the board.
```

An arrow can escape only when its complete movement path toward the board boun
is unobstructed.

The player must determine the correct order.

Example:

Arrow A points RIGHT.

If another arrow occupies any cell in the swept movement path of Arrow A,
Arrow A cannot escape.

If the path is clear:
- player taps Arrow A
- Arrow A animates smoothly toward the right
- it exits the board
- its occupied cells become empty
- newly available arrows can now escape

The puzzle is therefore fundamentally about:

1. spatial reasoning
2. dependency detection
3. ordering
4. forward planning
5. recognizing which arrow is currently "free"

The game must feel simple to understand but increasingly difficult to solve.

CORE DESIGN PRINCIPLE:

"Easy to understand.
Hard to master.
Satisfying to solve."

---

1. GAME BOARD

Implement a logical grid independent from rendering.

Board sizes should support:

4x4
5x5
6x6
7x7
8x8
10x10
12x12
15x15
20x20+

The rendering system must NOT assume a fixed board size.

Create:

BoardModel

Properties:

- width
- height
- arrows
- boardBounds
- levelID
- difficulty
- theme
- metadata

Each Arrow contains:

ArrowModel

- id
- color
- cells: [GridPoint]
- direction: Direction
- headCell
- isActive
- isEscaping
- isRemoved
- difficultyMetadata

Direction:

```
enum Direction {
    up
    down
    left
    right
}
```

GridPoint:

```
struct GridPoint {
    let row: Int
    let column: Int
}
```

The board model must be completely independent from SpriteKit nodes.

---

2. ARROW REPRESENTATION

An arrow is NOT just a single straight line.

Represent it as a connected set of grid cells.

Example:

```
[
  (2,2),
  (2,3),
  (2,4),
  (3,4),
  (4,4)
]
```

with direction RIGHT.

The visual renderer converts this logical shape into:

- horizontal line segments
- vertical line segments
- corner joints
- arrowhead

The arrow must look like one continuous thin path.

Use a consistent line thickness.

The arrowhead should visually sit at the final/head cell.

Do not use individual independent sprites for every segment unless there is a performance reason.

Prefer rendering an arrow as one SKShapeNode/path where possible.

---

3. ARROW GEOMETRY

Create:

ArrowGeometryBuilder

Responsibilities:

- convert grid cells to screen coordinates
- create orthogonal path
- determine horizontal/vertical segments
- create corner joins
- create arrowhead
- create hit-test area
- create escape trajectory
- create selection highlight
- create hint visualization

The visual line should be slightly rounded at corners.

Use antialiasing where appropriate.

The line should remain visually crisp on Retina displays.

---

4. CORE COLLISION / ESCAPE ALGORITHM

This is the most important part of the game.

DO NOT implement gameplay collision using approximate SpriteKit physics.

The puzzle must use deterministic GRID LOGIC.

Create:

PuzzleEngine

Main functions:

```
canEscape(arrowID)
attemptEscape(arrowID)
removeArrow(arrowID)
availableArrows()
isSolved()
reset()
undo()
```

When checking an arrow:

1. Determine its movement direction.
2. Determine its current occupied cells.
3. Simulate translating the entire arrow by one grid cell in that direction.
4. Check whether any translated cell overlaps another active arrow.
5. Continue until the arrow leaves the board.
6. If collision occurs before exiting:
   return BLOCKED.
7. Otherwise:
   return CAN_ESCAPE.

Use efficient occupancy lookup.

Maintain:

```
Dictionary<GridPoint, ArrowID>
```

or an equivalent efficient structure.

Do not repeatedly scan every arrow and every cell unnecessarily.

Expected complexity:

Basic escape check:
$O(pathLength * occupiedCells)$

Optimize this where practical.

For large boards, use spatial occupancy maps.

---

5. IMPORTANT: SWEPT COLLISION

The entire arrow moves.

Do NOT check only the arrowhead.

Do NOT check only the destination.

The body of the arrow must also be considered.

Example:

Arrow shape:

```
AAAA
   A
   A
```

If the arrow moves RIGHT, the entire shape translates RIGHT.

If any part of the translated shape overlaps another arrow,
the movement is blocked.

Implement this deterministically.

---

6. DEPENDENCY GRAPH

Create an optional analytical layer:

PuzzleDependencyGraph

For every arrow:

A -> B

means:

A cannot escape until B has moved.

Build the dependency graph using the same collision rules.

This will be useful for:

- hint generation
- puzzle validation
- difficulty calculation
- level generation
- analytics
- debugging
- solver mode

Do not make the dependency graph the sole source of truth.
PuzzleEngine remains authoritative.

---

7. PUZZLE SOLVER

Create:

PuzzleSolver

The solver should determine whether a puzzle is solvable.

Use:

DFS / BFS / backtracking

State:

Set<ArrowID>

representing removed arrows.

At every state:

1. Find all currently escapable arrows.
2. Try removing each one.
3. Recursively solve.
4. Backtrack when necessary.

Return:

SolutionResult

- isSolvable
- moveSequence
- numberOfSolutions
- minimumMoves
- branchingFactor
- estimatedDifficulty

The solver is primarily an offline development tool.

Do not run an expensive solver on the main game thread.

---

8. HINT SYSTEM

Implement multiple hint levels.

Hint Level 1:
Highlight one arrow that can currently escape.

Hint Level 2:
Highlight the arrow and show its escape direction.

Hint Level 3:
Show the first few correct moves.

Hint Level 4:
Automatically perform one correct move.

Hints must NEVER randomly select an arrow.

They must use PuzzleSolver / PuzzleEngine.

UI:

Hint button with remaining hint count.

When hint is activated:

- subtly pulse the correct arrow
- optionally draw a translucent trajectory
- show a small instructional message
- never cover the puzzle

Hint animation should take approximately 0.6–1.0 seconds.

---

9. "LINES" / VISUAL GUIDE FEATURE

Implement a dedicated Lines utility.

This is an important quality-of-life feature.

The player can toggle:

SHOW LINES

The feature should make the board easier to understand by visually exposing the logical movement/trajectory information.

Possible visualization:

- subtle dotted projection from arrowhead toward board edge
- collision/blocker indication
- escape lane
- directional guide

Do NOT make the lines visually overpower the puzzle.

Use:

LineGuideRenderer

States:

OFF
SUBTLE
FULL

Default:
SUBTLE

Animation:

When enabled:
fade lines in over ~200ms.

When disabled:
fade lines out over ~150ms.

Allow the setting to be remembered.

---

10. TOUCH INTERACTION

Touch interaction is critical.

Implement:

Tap
Drag
Long Press only if useful

Primary interaction:
TAP an arrow.

Use generous hit areas.

The visual line may be 2–4 px wide,
but the touch target should be significantly larger.

Example:
visual line = 3 px
hit target = 28–40 px

This prevents accidental taps.

Important:
Do not let a nearby arrow steal touches.

Hit testing priority:

1. exact arrow body
2. arrowhead
3. nearest valid arrow within tolerance

If two arrows overlap visually,
use deterministic selection rules.

Never rely on SpriteKit's default node hit testing alone.

---

11. TAP FEEDBACK

When the player taps an arrow:

If valid:

- tiny scale pulse
- arrow brightens
- movement begins immediately
- smooth escape animation
- subtle sound
- subtle haptic

If blocked:

- arrow performs a small shake
- blocker can optionally flash briefly
- heart/mistake feedback occurs
- sound effect
- haptic feedback

Do NOT use aggressive screen shake.

The game should feel calm.

---

12. ESCAPE ANIMATION

Valid arrow:

1. selected arrow brightens
2. slight anticipation scale
3. arrow begins moving
4. moves smoothly in its direction
5. accelerates slightly
6. leaves board
7. fades/scales subtly
8. disappears
9. board updates
10. newly available arrows become visually clear

Recommended timing:

anticipation:
0.08–0.12 sec

movement:
0.35–0.75 sec

exit:
0.15–0.25 sec

Use easing.

Avoid linear animation.

Prefer:
easeIn

easeOut
or a custom cubic timing curve.

---

13. CHAIN REACTION / SOLVE FEEL

When multiple arrows become available after a move,
do NOT automatically move them.

Instead:

- subtly highlight newly available arrows
- optionally pulse them once
- let the player choose

The player should retain control.

On the final arrow:

Use a stronger completion animation.

Example:

- final arrow exits
- board pauses
- small particle burst
- completion sound
- stars appear
- level completion card slides in

---

14. FAILURE / MISTAKE SYSTEM

The game uses hearts/lives as a light consequence system.

Default:

3 hearts.

A blocked arrow tap:

- removes one heart
- plays feedback
- does not remove the arrow

If hearts reach zero:

show:

"Out of Hearts"

Options:

- Retry

- Undo last mistake
- Watch reward ad
- Use premium continuation
- Return to level select

IMPORTANT:
The game should remain playable offline.

For the first version, implement the core system without ads.

Create an abstraction:

LifeService

Later this can connect to an ad provider.

---

15. UNDO

Implement full undo.

Every successful arrow removal creates a snapshot:

GameStateSnapshot

Store:

- removed arrows
- active arrows
- hearts
- moves
- mistake count

Undo should restore the previous puzzle state.

Maximum history:

20 states initially.

Undo must animate the restored arrow back into place.

Do NOT simply reload the level.

---

16. RESET

Reset returns the puzzle to its original state.

Confirmation only when appropriate.

Do not unnecessarily show confirmation for a simple reset.

---

17. LEVEL COMPLETION

Track:

- moves
- mistakes
- hints used
- time
- hearts remaining
- completion status

Calculate:

1-star
2-star
3-star
4-star
5-star

Do not make the exact scoring arbitrary.

Suggested model:

Base score = puzzle difficulty

Penalties:
- mistakes
- hints
- undo usage

Bonus:
- remaining hearts
- optimal solution
- no mistakes

The user should be able to replay completed levels.

---
18. LEVEL DIFFICULTY

Difficulty tiers:

BEGINNER
EASY
MEDIUM
HARD
VERY_HARD
EXPERT
NIGHTMARE

Difficulty should be based on puzzle properties rather than simply board size.

Calculate:

- number of arrows

- average arrow length
- longest arrow
- dependency depth
- dependency graph branching
- number of initially available arrows
- number of dead-end choices
- minimum solution length
- number of misleading moves
- density
- symmetry
- number of intersections
- board size

Create:

DifficultyAnalyzer

Return:

DifficultyScore 0–100.

---

19. LEVEL GENERATOR

Create:

LevelGenerator

Generate puzzles algorithmically.

Pipeline:

1. Create empty grid.
2. Generate valid arrow paths.
3. Ensure paths do not overlap.
4. Assign directions.
5. Verify every arrow can eventually escape.
6. Run PuzzleSolver.
7. Reject unsolvable puzzles.
8. Calculate difficulty.
9. Store valid puzzle.

The generator must create ORIGINAL levels.

Never copy levels from another game.

---

20. LEVEL GENERATION STRATEGIES

Support:

A. Hand-authored levels

JSON files.

B. Procedural levels

Random generation.

C. Solver-assisted levels

Generate random board,
then validate with solver.

D. Difficulty-targeted generation.

Example:

```
generateLevel(
    boardSize: 8,
    targetDifficulty: 60
)
```

The generator should continue generating until
the requested difficulty range is reached.

---

21. LEVEL DATA FORMAT

Use JSON.

Example:

```
{
    "id": "level_0001",
    "width": 6,
    "height": 6,
    "difficulty": 18,
    "arrows": [
        {
            "id": "A1",
            "direction": "right",
            "cells": [
                {"row": 1, "column": 1},
                {"row": 1, "column": 2},
                {"row": 1, "column": 3}
            ]
        }
    ]
}
```

The format must be easy for humans to edit.

---

22. LEVEL EDITOR

Create a DEBUG-ONLY level editor.

Features:

- choose board size
- draw arrow
- erase arrow
- choose direction
- validate
- solve
- show dependency graph
- calculate difficulty
- preview
- export JSON

This is extremely important for building thousands of levels.

Do not require manually editing JSON for every level.

---

23. MAIN GAME SCREEN

Visual hierarchy:

TOP:
- hearts
- level indicator
- optional hint count

CENTER:
- puzzle board

BOTTOM:
- Undo
- Reset
- Hint
- Lines

The puzzle should occupy most of the screen.

Avoid unnecessary UI.

Background:
warm/light neutral by default.

Use a very clean minimalist visual system.

---

24. GAME SCREEN LAYOUT

Use safe-area-aware layout.

Example:

Top safe area:
10–16%

Header:
8%

Puzzle:
65–72%

Controls:
10–14%

Do not hardcode pixels.

Calculate board size dynamically:

availableWidth
availableHeight

boardDimension =
min(availableWidth, availableHeight)

Center board.

For very large boards:
support pinch zoom and pan.

Minimum zoom:
0.75x

Maximum zoom:
2.5x

Double tap:
reset zoom.

---

25. PINCH ZOOM

Implement zoom only when useful.

Small puzzles:
no zoom necessary.

Large puzzles:
pinch-to-zoom.

Important:
Touch coordinates must correctly map:

screen -> scene -> board -> grid

Create:

CoordinateMapper

Functions:

screenPointToGridPoint()
gridPointToScreenPoint()

This must be thoroughly unit tested.

---

26. UI SCREENS

Implement these screens:

1. Splash
2. Home
3. Continue Game
4. Level Select
5. Difficulty Select
6. Puzzle
7. Level Complete
8. Daily Challenge
9. Daily Calendar
10. Tournament
11. Statistics
12. Achievements
13. Settings
14. Help / How to Play
15. Theme Selection
16. Hint Shop placeholder
17. Hearts / Continue modal

---

27. HOME SCREEN

Minimal.

Logo/title at top.

Primary CTA:

CONTINUE

Secondary:

PLAY

DAILY

Below:

Current streak
Progress
Best difficulty

Bottom navigation:

Home
Levels
Daily
Stats
Settings

Do not overload the home screen.

---

28. LEVEL SELECT

Organize levels into chapters/worlds.

Example:

Chapter 1:
First Steps

Chapter 2:
Crossroads

Chapter 3:
Tangles

Chapter 4:
Deep Maze

Chapter 5:
Mind Bender

Chapter 6:
Nightmare

Each chapter contains:

Level number
difficulty
stars
completion state

Locked levels display lock.

Completed levels display stars.

---

29. DAILY CHALLENGE

One puzzle per day.

Use deterministic seed:

YYYY-MM-DD

The same date produces the same puzzle for all players.

Store:

- completed
- score
- stars
- mistakes
- time
- hints

Provide historical calendar.

Completed previous daily puzzles remain replayable.

---

30. TOURNAMENT

Implement an offline/local tournament architecture first.

Every tournament has:

- ID
- startDate
- endDate
- puzzle
- scoringRules

Score:

base puzzle value
+
remaining hearts bonus
+
low mistake bonus
+
optimal move bonus

Leaderboard service should be abstract:

LeaderboardService

Initially:
LocalLeaderboardService

Later:
CloudLeaderboardService.

---

31. STREAK SYSTEM

Track daily activity.

Properties:

currentStreak
longestStreak
lastCompletedDate

Handle date changes safely.

Do not punish users for timezone changes.

Use local calendar dates.

Show:

"7 Day Streak"

with subtle animation.

---

32. ACHIEVEMENTS

Examples:

First Escape
10 Puzzles
50 Puzzles
100 Puzzles
No Mistakes
Perfect 10
Nightmare Solver
Daily Player
7 Day Streak
30 Day Streak

Create:

AchievementService

All achievements should be data-driven.

---

33. THEMES

Create a theme system.

Theme:

- backgroundColor
- arrowColor
- guideColor
- accentColor
- completedColor
- blockedColor
- UI colors

Themes:

Classic
Midnight
Paper
Ocean
Forest
Sunset
Monochrome

Do NOT hardcode colors throughout the codebase.

---

34. ACCESSIBILITY

Support:

- Reduce Motion
- High Contrast
- Larger UI text
- Color-blind-friendly themes
- Haptic toggle
- Sound toggle

Do not make color the only indication of state.

Use:

shape
animation
highlight
icons

along with color.

---

35. AUDIO

Create:

AudioManager

Sounds:

arrowTap
blocked
arrowEscape
undo
hint
levelComplete
buttonTap
dailyComplete
achievement

Music:

soft ambient loop.

Allow:

Music ON/OFF
SFX ON/OFF
Haptics ON/OFF

The audio must be subtle.

---

36. HAPTICS

Use UIKit/Core Haptics where appropriate.

Tap:
light

Blocked:
medium

Successful escape:
light

Level completion:
success pattern

Do not spam haptics.

---

37. PARTICLES

Use very limited particles.

Level completion:

small sparkle particles.

Achievement:

small celebratory effect.

Do not turn the minimalist game into a particle-heavy game.

---

38. ANIMATION SYSTEM

Create reusable:

AnimationManager

Functions:

```
animateArrowEscape()
animateArrowBlocked()
animateArrowSelection()
animateHint()
animateLevelComplete()
animateHeartLoss()
animateButtonPress()
```

Avoid hardcoded animation code scattered everywhere.

---

39. VISUAL POLISH

The visual identity should be:

minimal
calm
clean
high contrast
thin geometric lines
subtle shadows
small rounded UI elements
generous whitespace

The board is the hero.

Never allow menus to visually dominate gameplay.

---

40. ARROW RENDERING DETAILS

Use:

thin strokes
rounded line caps
rounded joins

Arrowhead:

small triangular geometry.

Arrow body:

approximately 2–4 screen points depending on device scale.

Selection:

increase brightness and optionally stroke width.

Blocked:

briefly tint the arrow and animate a subtle shake.

Available:

optional subtle pulse.

---

41. RESPONSIVE LAYOUT

Must support:

iPhone SE
iPhone standard
iPhone Pro Max
iPad Mini
iPad
iPad Pro

Never assume a specific screen resolution.

Use normalized layout calculations.

---

42. PERFORMANCE

Target:

60 FPS.

Avoid:

- unnecessary SKNode creation
- rebuilding the entire board after every tap
- physics simulation
- repeated expensive path calculations
- scanning every arrow for every frame

Only update changed arrows.

For very large levels:

- spatial occupancy map
- dirty rendering
- cached geometry

---

43. GAME STATE ARCHITECTURE

Create:

GameSession

Contains:

currentLevel

gameState
hearts
moves
mistakes
hints
undoStack
startTime
elapsedTime

GameState:

playing
paused
completed
failed

Use clear state transitions.

---

44. ARCHITECTURE

Use separation of concerns.

Suggested structure:

```
ArrowFlow/
    App/
    Models/
    Engine/
    Solver/
    LevelGenerator/
    LevelEditor/
    Rendering/
    Animation/
    Input/
    UI/
    Persistence/
    Audio/
    Haptics/
    Services/
    Utilities/
    Resources/
    Levels/
    Tests/
```

Important classes:

PuzzleEngine
PuzzleSolver
LevelGenerator
DifficultyAnalyzer
ArrowRenderer
BoardRenderer
CoordinateMapper
InputManager

AnimationManager
AudioManager
HapticsManager
HintService
LifeService
AchievementService
DailyChallengeService
TournamentService
ProgressService
PersistenceService
ThemeManager

---

45. PROTOCOL-BASED SERVICES

Use protocols where external implementations may change.

Example:

```
protocol PersistenceService
protocol AudioService
protocol HapticsService
protocol LevelProvider
protocol LeaderboardService
protocol AdService
protocol AnalyticsService
```

Use dependency injection.

Do not create massive singleton architecture.

Singletons are acceptable only for genuinely global infrastructure.

---

46. TESTING

Create extensive unit tests.

Test:

- simple escape
- blocked arrow
- multiple blockers
- curved arrow
- arrow touching edge
- arrow completely inside board
- multiple valid arrows
- no valid arrows
- puzzle completion
- reset
- undo
- hints
- solver
- level generation

- difficulty
- daily seed
- streak
- coordinate mapping
- zoom
- touch hit testing

Important:

Create 100+ deterministic puzzle-engine test cases.

---

47. PROPERTY-BASED TESTS

For every generated valid puzzle:

ASSERT:

- every arrow is inside board
- no two arrows overlap initially
- every generated puzzle is solvable
- solver returns a valid sequence
- executing solver sequence completes puzzle
- no illegal move appears in solution

This should be part of development tooling.

---

48. DEBUG OVERLAY

Add developer-only debug overlay.

Show:

FPS
board size
arrow count
selected arrow
grid coordinates
dependency graph
available arrows
solver result
difficulty score
memory state

Toggle using:

three-finger tap
or DEBUG menu.

---

49. CHEAT / DEBUG COMMANDS

Developer-only:

Solve Level
Next Level
Previous Level
Reset Hearts
Add Hint
Show Collision
Show Occupancy Grid
Show Dependencies
Show Solution
Generate Random Level
Generate Difficulty 50
Generate Difficulty 90

---

50. SAVE SYSTEM

Persist:

current level
completed levels
stars
daily progress
streak
settings
themes
achievements
statistics
hints
hearts

Use Codable.

Version saved data.

Example:

SaveDataVersion = 1

Future migrations must be possible.

---

51. SETTINGS

Settings:

Sound
Music
Haptics
Reduced Motion
High Contrast
Show Lines
Auto Hint

Theme
Language
Notifications
Privacy

Keep settings minimal.

---

52. ONBOARDING

First launch:

Screen 1:

"Find the arrow with a clear path."

Screen 2:

"Tap it to escape."

Screen 3:

"Every escape opens new paths."

Screen 4:

"Think ahead."

Then first tutorial level.

Tutorial should visually demonstrate the mechanic.

Do not dump text on the player.

---

53. GAMEPLAY FEEL

The desired loop:

SEE BOARD
↓
SCAN
↓
FIND AVAILABLE ARROW
↓
TAP
↓
WATCH IT ESCAPE
↓
BOARD OPENS
↓
SCAN AGAIN
↓
SOLVE

↓
CELEBRATE
↓
NEXT LEVEL

The game should feel meditative.

No unnecessary timers in the main mode.

---

54. QOL FEATURES

Implement:

- Undo
- Restart
- Hint
- Show Lines
- Pinch Zoom
- Pan
- Reduced Motion
- Haptics toggle
- Sound toggle
- Dark theme
- Color-blind-friendly theme
- Level replay
- Daily puzzle history
- Auto-save
- Resume current puzzle
- Confirmation before losing progress
- Larger touch targets
- Optional blocker highlighting
- Statistics
- Best score
- Streak
- Achievements

---

55. OPTIONAL ADVANCED FEATURES

After core game is stable:

Add:

- timed mode
- challenge mode
- endless mode
- daily tournament
- online leaderboard
- cloud save
- player profiles
- custom level sharing
- level code import/export
- community levels

- weekly challenge
- seasonal themes

Do NOT implement these before the core puzzle is perfect.

---

56. ASSET GENERATION

The game is primarily procedural/vector based.

Do NOT generate the board as raster images.

Use code-generated geometry for:

- arrows
- lines
- grid
- arrowheads
- highlights
- buttons where possible

Only generate raster assets for:

- app icon
- onboarding illustrations
- achievement icons
- theme artwork
- background decoration
- promotional artwork

---

57. ORIGINAL ASSET PROMPTS

Create an original visual identity.

APP ICON PROMPT:

"Minimalist abstract arrow escape puzzle icon,
single geometric arrow traveling through a clean maze,
flat vector design,
high contrast,
premium mobile game aesthetic,
simple silhouette,
no text,
no existing brand resemblance,
square app icon,
centered composition,
clean negative space"

ONBOARDING ART:

"Minimalist abstract geometric puzzle board,
thin flowing arrow paths,
one highlighted escape route,

soft modern mobile game aesthetic,
white background,
subtle blue accent,
vector illustration,
no text,
original artwork"

ACHIEVEMENT ICONS:

"Set of minimalist geometric puzzle achievement icons,
arrow motifs,
stars,
maze symbols,
clean vector design,
consistent visual language,
mobile game UI,
transparent background,
no text"

THEME ART:

"Abstract geometric maze pattern,
thin continuous lines,
minimalist vector artwork,
subtle depth,
calm puzzle-game aesthetic,
seamless background"

---

58. TWEEN / ANIMATION ASSET GENERATION

Do not create separate animation sprites for normal arrow movement.

Generate movement procedurally.

Animations should be deterministic.

Create reusable animation presets:

ArrowEscapeFast
ArrowEscapeNormal
ArrowEscapeRelaxed
ArrowBlocked
ArrowHint
LevelComplete
AchievementUnlock

Allow tuning through constants.

Example:

AnimationConfig

- duration
- delay

- easing
- scale
- opacity
- rotation
- bounce

All tunable centrally.

---

59. SCREEN FLOW

APP LAUNCH

Splash
↓
Home
↓
Continue
↓
Puzzle
↓
Level Complete
↓
Next Level

Alternative:

Home
↓
Levels
↓
Chapter
↓
Level
↓
Puzzle

Daily:

Home
↓
Daily
↓
Calendar
↓
Daily Puzzle
↓
Result

Stats:

Home
↓
Statistics
↓

Achievements

Settings:

Home
↓
Settings
↓
Theme / Accessibility / Audio

---

60. FIRST IMPLEMENTATION MILESTONE

DO NOT implement the entire application at once.

First build ONLY:

1. BoardModel
2. ArrowModel
3. GridPoint
4. Direction
5. PuzzleEngine
6. ArrowRenderer
7. BoardRenderer
8. CoordinateMapper
9. Tap interaction
10. Escape collision detection
11. Escape animation
12. Level completion

Create 10 handcrafted levels.

No menus yet.

The first milestone is successful only when:

- app launches
- puzzle appears
- player can tap arrows
- blocked arrows correctly fail
- valid arrows escape
- board updates
- final arrow completes level
- reset works

---

61. SECOND MILESTONE

Add:

- hearts
- undo
- hints
- lines

- restart
- level progression
- 50 levels
- save state
- completion stars

---

62. THIRD MILESTONE

Add:

- level select
- chapters
- difficulty
- themes
- settings
- audio
- haptics
- onboarding
- achievements
- statistics

---

63. FOURTH MILESTONE

Add:

- procedural level generation
- solver
- difficulty analyzer
- level editor
- 1000 generated candidate levels
- validation pipeline

---

64. FIFTH MILESTONE

Add:

- daily puzzle
- streak
- tournament
- leaderboard abstraction
- advanced modes

---

65. CODE QUALITY RULES

Rules:

1. No giant ViewController.
2. No giant GameScene.

3. No business logic inside SKShapeNode.
4. No hardcoded screen coordinates.
5. No hardcoded level logic.
6. No random unsolvable levels.
7. No physics-based puzzle collision.
8. No force unwraps unless genuinely impossible.
9. Use descriptive names.
10. Keep functions small.
11. Write tests before optimizing.
12. Keep game logic deterministic.
13. Keep rendering separate from state.
14. Keep persistence separate from gameplay.
15. Use protocols where dependencies may change.

---

66. CODING AGENT WORKFLOW

When starting:

STEP 1:
Inspect the existing repository.

STEP 2:
Identify current architecture.

STEP 3:
Do NOT delete working code without explaining why.

STEP 4:
Create a short implementation plan.

STEP 5:
Implement only the first milestone.

STEP 6:
Build.

STEP 7:
Run tests.

STEP 8:
Fix compilation errors.

STEP 9:
Run the app.

STEP 10:
Only then proceed to the next milestone.

After each milestone report:

- files created
- files changed
- architecture decisions
- tests added

- current functionality
- known limitations
- next milestone

Never silently skip compilation or tests.

---

67. IMPORTANT GAMEPLAY VALIDATION

Before declaring the core engine complete,
manually validate these examples.

CASE 1:

One arrow pointing right with clear path.

Expected:
escape.

CASE 2:

One arrow pointing right blocked by another arrow.

Expected:
blocked.

CASE 3:

A is blocked by B.
B is blocked by C.
C is free.

Expected:
C -> B -> A.

CASE 4:

Multiple free arrows.

Expected:
any valid free arrow may be selected.

CASE 5:

Arrow has an L-shaped body.

Expected:
entire arrow moves as one piece.

CASE 6:

Arrow body would collide even though arrowhead path is clear.

Expected:
blocked.

CASE 7:

Undo after successful move.

Expected:
arrow restored exactly.

CASE 8:

Undo after multiple moves.

Expected:
correct historical state restored.

CASE 9:

Last arrow escapes.

Expected:
level complete.

CASE 10:

Generated puzzle.

Expected:
solver confirms solvability.

---

68. FINAL PRODUCT STANDARD

The finished game should feel like a premium minimalist puzzle game.

The player should immediately understand:

"Find an arrow that has a clear path.
Tap it.
It leaves.
Repeat."

But after 20–50 levels,
the player should begin thinking:

"If I remove this arrow,
I unlock that one,
but then that creates a dependency with these three..."

That transition from simple interaction to deep planning
is the core of the game.

DO NOT sacrifice this core mechanic for excessive UI,
monetization or visual effects.

BUILD THE PUZZLE ENGINE FIRST.

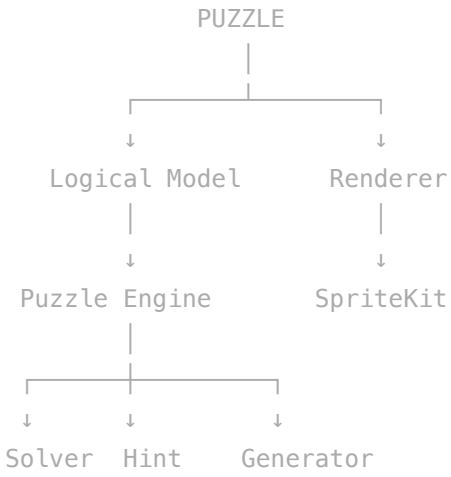
```
Then build everything around it.
```

I would make one important architectural change

For this particular game, **don't let Cursor build it as a normal SpriteKit physics game**.

The screenshots make it look visual, but underneath it should be a **DSA problem**.

Think of the game as:



The game renderer should basically be a visualization of the mathematical model.

That will make your project **much easier to debug**.

The DSA behind the game

This is actually a very nice DSA project.

1. Grid

At the lowest level:

```
struct GridPoint: Hashable {
    let row: Int
    let column: Int
}
```

Then:

```
var occupancy: [GridPoint: UUID]
```

Example:

```
. . . . . .
. A A A . .
. . . B B .
. . . B . .
. . . . . .
```

becomes conceptually:

```
(1,1) -> A
(1,2) -> A
(1,3) -> A

(2,3) -> B
(2,4) -> B
(3,3) -> B
```

This makes collision detection straightforward.

2. Arrow as a graph/path

An arrow isn't necessarily:

```
→→→→
```

It can be:

```
┌──┐
   │
   │
   └─→
```

So internally:

```
A A A
    A
    A
    A A
```

is simply a set/path of occupied cells.

3. Escape algorithm

Suppose:

```
A A A →
```

and another arrow is here:

```
A A A →    B B
```

The engine checks:

```
position + direction
```

repeatedly.

Conceptually:

```swift
var translatedCells = arrow.cells

while !outsideBoard(translatedCells) {

    translatedCells = translatedCells.map {
        $0.moved(in: arrow.direction)
    }

    if collides(translatedCells) {
        return .blocked
    }
}

return .canEscape
```

That is the core mechanic.

4. Dependency graph

This is where it gets interesting.

Imagine:

```
A → blocked by B
B → blocked by C
C → free
```

You can represent:

```
A
↓
B
```

```
↓
C
```

Therefore the solution is:

```
C
B
A
```

This gives you a natural **graph problem**.

You can use this for:

- hints
- solver
- level generation
- difficulty calculation
- debugging
- "show me why this is blocked"

5. Solver

Your solver can do:

```
State 0

A B C D
```

Find all currently removable arrows:

```
C, D
```

Then branch:

```
    C
   / \
  A   D
  |   |
  D   C
  |
  ...
```

This is essentially:

DFS + backtracking over puzzle states.

That's also why I strongly recommend building the solver early.

It gives you an enormous advantage when generating levels.

6. Procedural level generation

Don't randomly draw arrows and hope the puzzle works.

Instead:

```
Generate
   ↓
Validate
   ↓
Solve
   ↓
Calculate difficulty
   ↓
Accept / Reject
```

For example:

```swift
generateLevel(targetDifficulty: 75)
```

could keep generating:

```
candidate #1 → difficulty 32 → reject
candidate #2 → difficulty 54 → reject
candidate #3 → difficulty 79 → accept
```

Now you can eventually generate **thousands of original puzzles**.

The "Lines" feature

This is something I'd actually improve over the original.

Instead of simply:

```
SHOW LINES
```

I'd implement three levels:

OFF

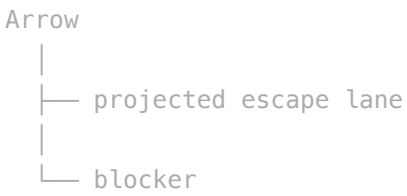
Pure puzzle.

SUBTLE

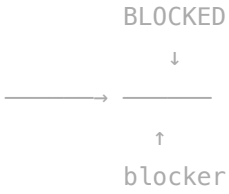
Show faint projected paths:

FULL

Show:



And when the user taps a blocked arrow:

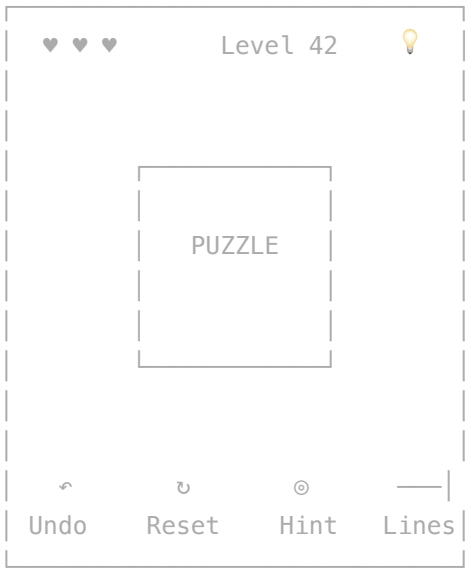


This becomes an extremely useful accessibility/QOL feature.

UI structure I'd recommend

The original game intentionally keeps the puzzle very clean. Its store description explicitly emphasizes minimalist design, smooth animations, no-pressure gameplay and focus on the puzzle. Google Play

So I'd use:



But don't literally copy their UI.

Make your own visual identity.

Screens you should build

I'd prioritize them like this:

MVP

1. Splash
2. Home
3. Level
4. Completion
5. Level select
6. Settings

V2

7. Daily
8. Daily calendar
9. Statistics
10. Achievements
11. Themes
12. Tutorial

V3

13. Tournament
14. Leaderboard
15. Challenge mode
16. Timed mode
17. Custom levels
18. Level editor

The current game's ecosystem already includes daily challenges and tournaments according to recent user reviews, so these are sensible retention systems to study—but I'd keep them out of your first build. Google Play +1

Animations to implement

Don't animate everything.

The game should feel **quiet**.

Arrow escape

```
tap
 ↓
highlight
 ↓
tiny anticipation
 ↓
accelerate
 ↓
slide
 ↓
exit
 ↓
fade
```

Blocked

```
tap
 ↓
tiny shake
 ↓
blocker highlight
 ↓
heart feedback
```

Hint

```
arrow
 ↓
soft glow
 ↓
trajectory appears
 ↓
pulse 2–3 times
 ↓
fade
```

Completion

```
last arrow
 ↓
escape
 ↓
board clears
 ↓
small particles
 ↓
stars
 ↓
completion card
```

Asset strategy

Do not generate the arrows as images.

That's important.

Generate:

```
arrow
line
corner
arrowhead
trajectory
highlight
grid
```

programmatically with SpriteKit.

This gives you:

- infinite resolutions
- different themes
- dynamic colors
- arbitrary board sizes
- arbitrary arrow shapes
- zoom support
- no asset explosion

You only need generated assets for things like:

```
App icon
Logo
Achievement icons
Onboarding art
Theme backgrounds
Marketing art
```

And don't ask the AI to generate 1,000 levels manually

Instead build:

```
LevelGenerator
        ↓
10,000 candidates
        ↓
PuzzleValidator
        ↓
PuzzleSolver
        ↓
```

```
DifficultyAnalyzer
        ↓
      JSON
        ↓
   LevelDatabase
```

Then you can have:

```
levels/
   beginner/
   easy/
   medium/
   hard/
   expert/
   nightmare/
   daily/
```

This is much more scalable.

One feature I'd add that the original doesn't do well

There are recent complaints about accidental touches and difficulty caused by small/ambiguous touch targets. Google Play +1

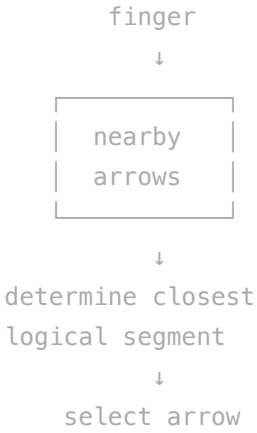
So I'd make **touch precision one of your differentiators**.

For example:

When finger touches near two arrows

Don't immediately choose whichever SpriteKit node happens to be hit.

Instead:



And potentially magnify the selected area slightly.

That makes the game feel much more polished.

Another feature I'd add: "Why blocked?"

This could be your signature QOL feature.

Player taps an unavailable arrow.

Instead of just:



show:

```
This arrow is blocked
```

Then animate:

```
YOUR ARROW
──────────→

          ▓▓▓▓
          BLOCKER
```

The blocker briefly pulses.

Then the UI disappears.

This teaches the player the game's logic without needing tutorials.

What I would NOT copy

Even though we're using this game as reference, don't copy:

- `Arrows` name
- logo
- exact colors
- exact UI
- exact level layouts
- exact artwork
- exact sounds
- exact animations
- exact progression numbers
- exact screenshots
- proprietary level data

Instead, use the **mechanical idea** and build a different presentation.

The Google Play listing currently identifies the original as **Arrows – Puzzle Escape by Lessmore GmbH**, with 100M+ installs shown on Google Play and an updated listing

dated September 1, 2026. Google Play

Recommended development order in Cursor

Don't paste the huge prompt and tell Codex:

> "Build everything."

That tends to produce a huge amount of mediocre code.

Instead use this sequence.

Prompt 1

```
Read the project specification.

Do NOT implement UI yet.

Implement only the logical puzzle engine.

Create:
GridPoint
Direction
ArrowModel
BoardModel
PuzzleState
PuzzleEngine
occupancy map
canEscape()
attemptEscape()
removeArrow()
reset()
undo()

Write comprehensive unit tests.

Do not use SpriteKit for the logic.

Build and run all tests.
```

Prompt 2

```
Now implement SpriteKit rendering.

Create:
BoardRenderer
ArrowRenderer
CoordinateMapper

Render the existing logical BoardModel.
```

```
Do not change PuzzleEngine.

The renderer must consume the model.

Add tap-to-select interaction.

Build and run.
```

Prompt 3

```
Implement the complete arrow escape animation.

The logical engine must remain authoritative.

Animation should never determine whether an arrow is allowed to escape.

Add:
selection
escape
blocked
completion animations.

Build and run.
```

Prompt 4

```
Implement:
hearts
undo
restart
hint
lines/trajectory guide

All must work against PuzzleEngine.

Write tests for each.
```

Prompt 5

```
Implement PuzzleSolver using DFS/backtracking.

Then implement:
DifficultyAnalyzer
LevelValidator
LevelGenerator.

Generate 1000 candidate puzzles and validate them.

Report:
solvability
minimum solution length
```

```
difficulty distribution
invalid puzzle count.
```

Prompt 6

```
Now build the complete UI:
Home
Level Select
Puzzle
Completion
Daily
Statistics
Achievements
Settings

Keep the gameplay board visually dominant.

Do not modify the core puzzle engine unless absolutely necessary.
```

Prompt 7

```
Now perform a production-quality polish pass.

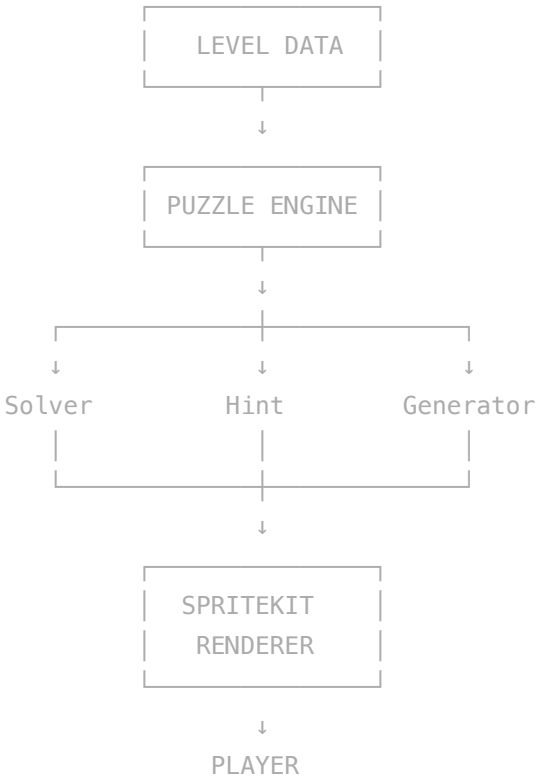
Check:
touch accuracy
animations
60 FPS
iPhone SE
iPhone Pro Max
iPad
safe areas
dark mode
reduced motion
accessibility
state restoration
memory usage
level loading
save corruption
orientation
background/foreground transitions.

Fix issues rather than merely reporting them.
```

The key takeaway

The interesting part of this project isn't actually the UI.

It's this:



If you build **that architecture first**, you'll have much more than a clone—you'll have a reusable **arrow-puzzle engine** capable of generating thousands of levels, different board sizes, multiple game modes, hints, tournaments, daily puzzles and potentially a level editor.

The current game's screenshots and description support exactly this minimalist, board-first direction: the puzzle dominates the screen, while hearts and other controls remain secondary. Google Play +2

And because you specifically mentioned **DSA**, this is actually an excellent project for that: **hash maps, grids, spatial indexing, graph dependencies, DFS/backtracking, state-space search, heuristics, procedural generation, and complexity analysis** all naturally fit into the game rather than being artificial interview exercises.

ChatGPT can make mistakes. Check important info.